

ABSTRACT

an **ai-powered** maze solver
visualizer for search algorithms

interactive exploration of maze solving

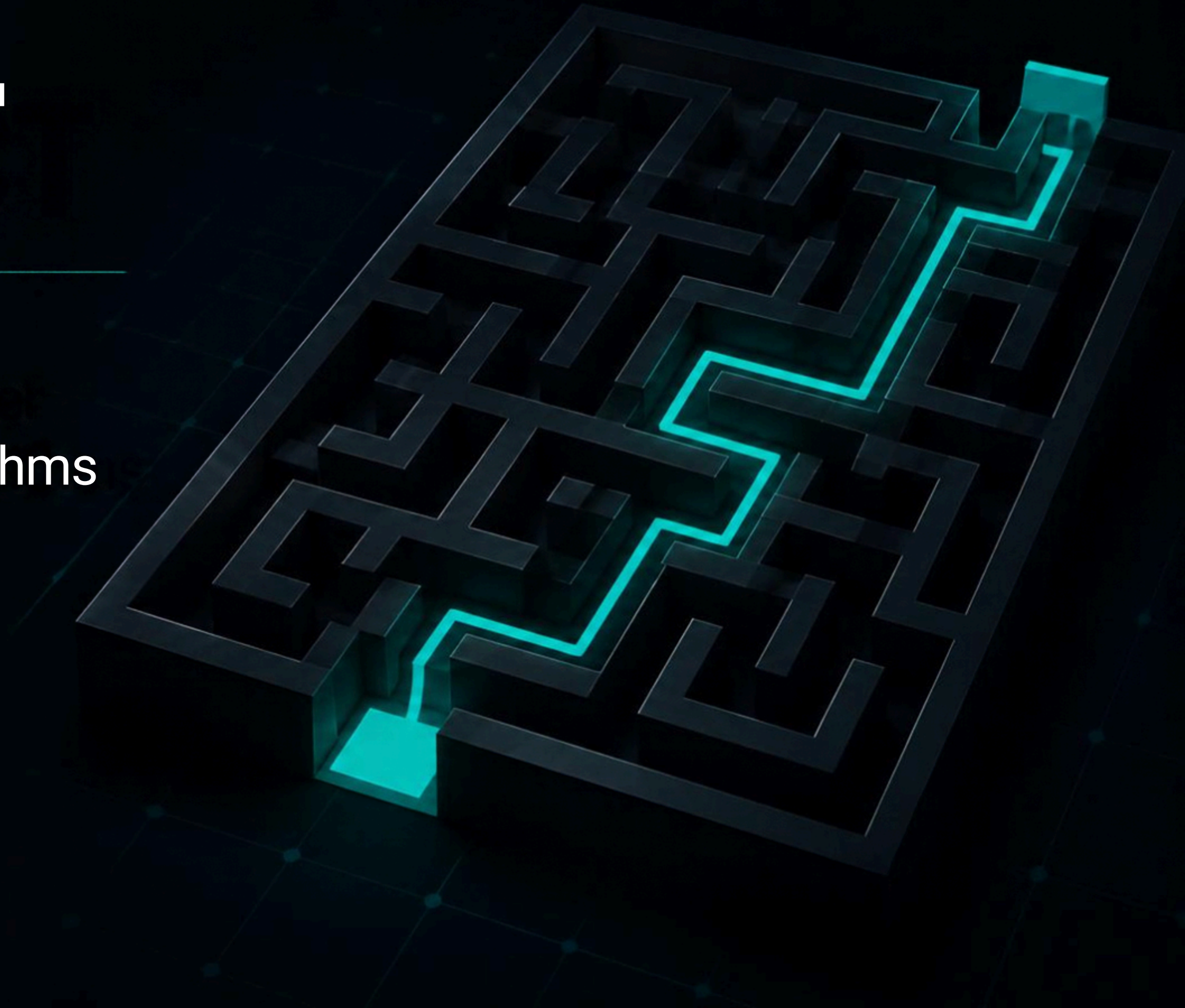
hill climbing and simulated annealing

compare search behavior step by step

TEAM

216240701508 — Shri Sarvendran PS

216240701469 — Sanjayram M

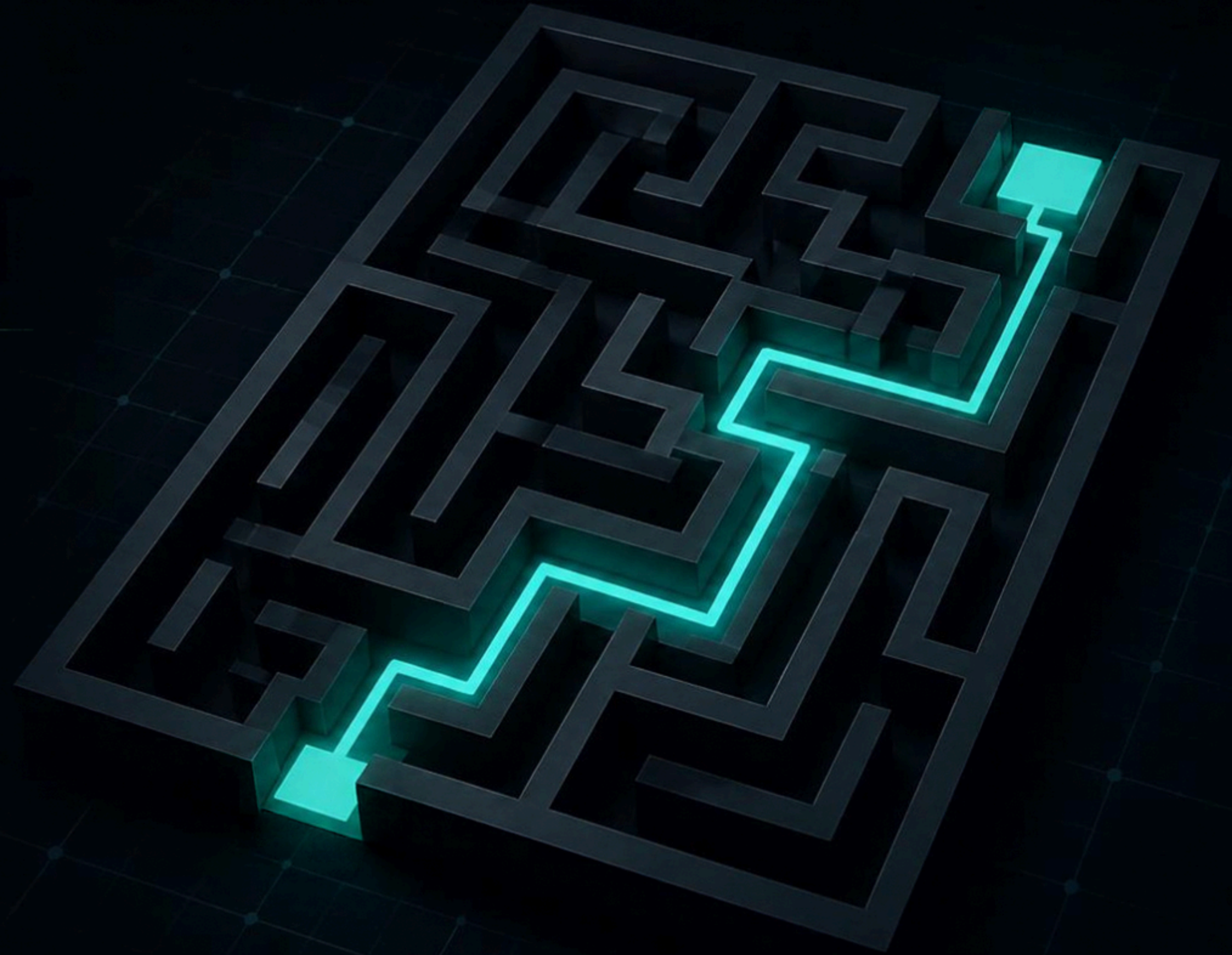


PROBLEM STATEMENT

Search algorithms
are **abstract** in theory

Students need to
observe search behavior

A maze
environment
provides an **intuitive
demonstration**



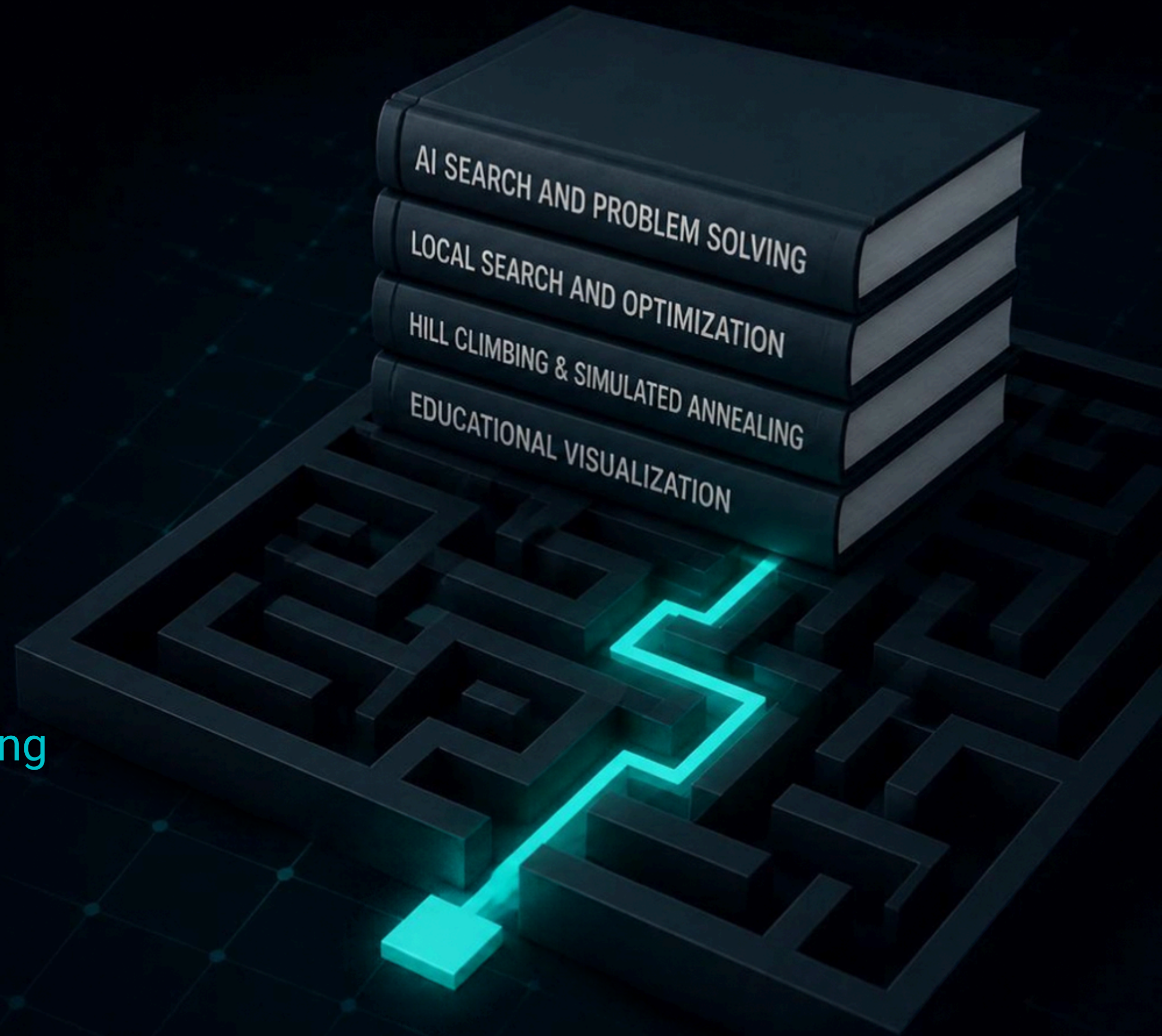
LITERATURE SURVEY

ai problem solving and search

local search and optimization

hill climbing and simulated annealing

educational visualization

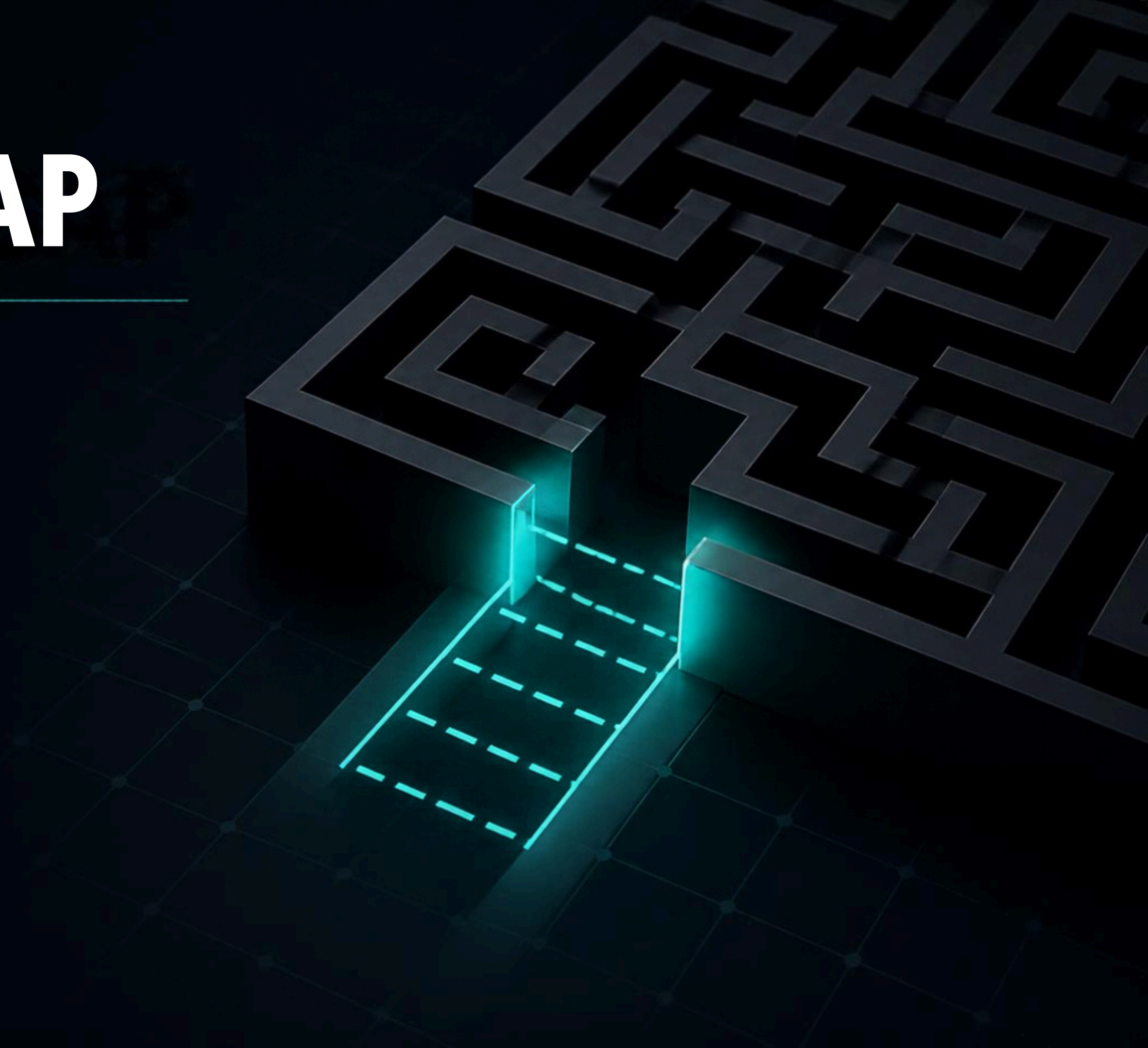


RESEARCH GAP

Search algorithms are
taught in isolation

Few tools visualize them
side by side

Students need an
interactive, maze-focused
learning experience



RESEARCH OBJECTIVE

01	visualize search behavior
02	implement two local-search algorithms
03	compare behavior on the same maze
04	build an educational maze game



PROPOSED WORK

SYSTEM ARCHITECTURE



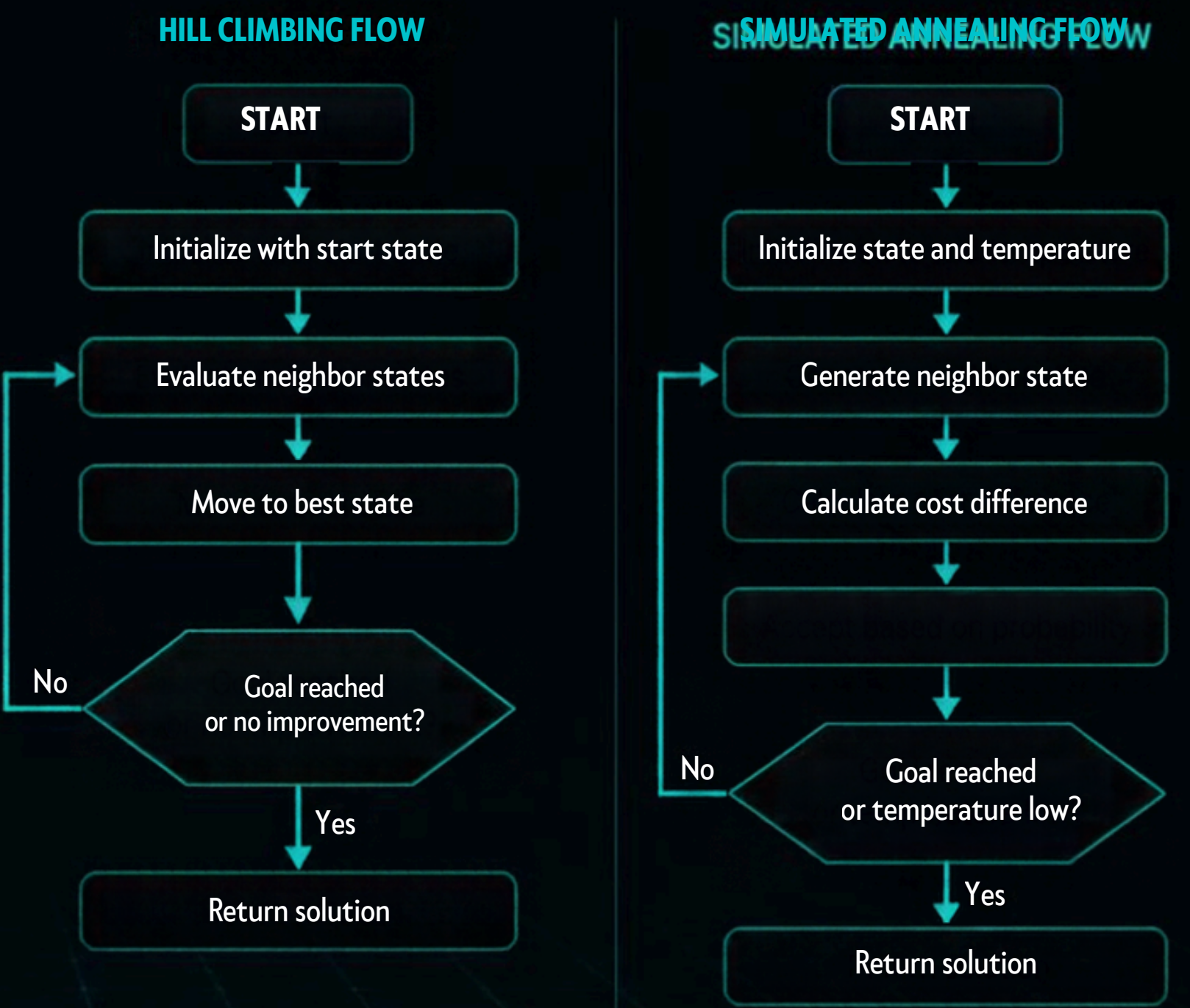
MAZE STATE

- Represents the agent position within the maze
- Transitions occur through valid neighboring cells
- Goal state is the exit cell

HEURISTIC

- Uses Manhattan distance to the goal
- Guides search toward promising states
- Reduces exploration and speeds convergence

ALGORITHM FLOWS



IMPLEMENTATION

Programming language & tools

Python implementation using core libraries for efficient algorithm execution and visualization

Maze / grid representation

Two-dimensional grid structure with walls, open cells, start, and goal states

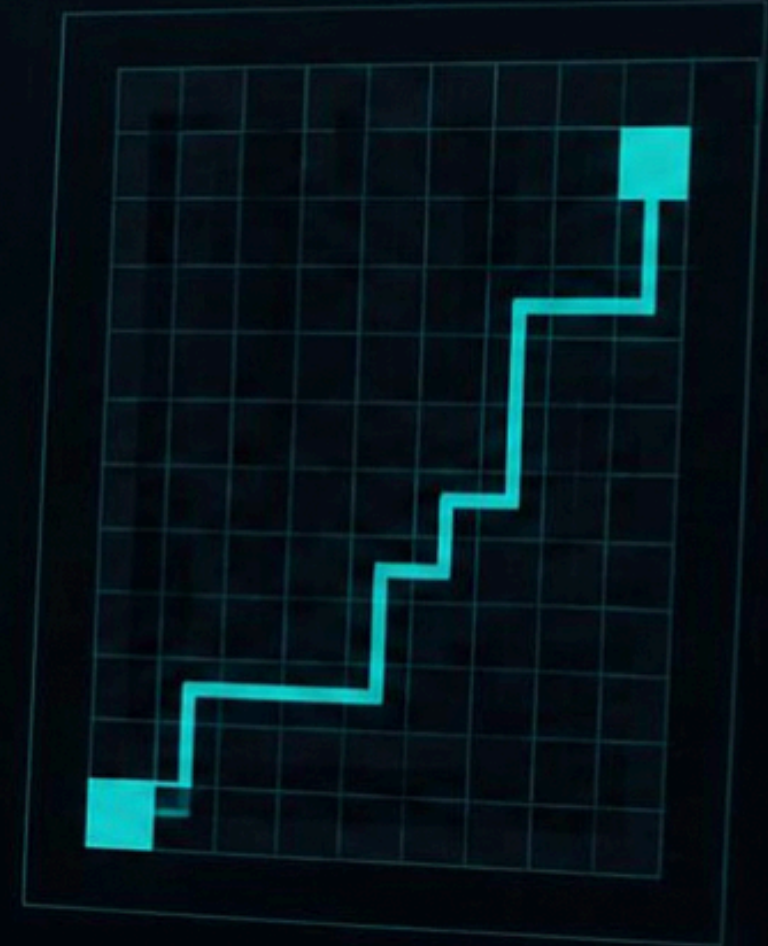
Heuristic & search states

Each state tracks position, cost, and parent to guide and reconstruct the optimal path

System configuration

Lightweight setup for smooth performance and interactive visualization

```
1 class Node:
2     def __init__(self, pos, g, h, parent=None):
3         self.pos = pos
4         self.g = g
5         self.h = h
6         self.parent = parent
7         self.f = g + h
8
9
10
11
12     def heuristic(a, b):
13         return abs(a[0]-b[0]) + abs(a[1]-b[1])
14
15
16     def neighbors(node, grid):
17         for n in valid_moves(node.pos, grid):
18             yield n
19
20
21     def reconstruct(node):
22         path = []
23         while node:
24             path.append(node.pos)
25             node = node.parent
26         return path[::-1]
```



OUTPUT

Visual results from the maze solver

MAZE GENERATION



Randomly generated
maze layout

VISITED CELLS



Cells explored during
search process

SOLVED PATH



Optimal path from start to
goal

ALGORITHM COMPARISON

BFS	Path Length 21 : 28	Nodes Visited 124
DFS	Path Length 24 : 6	Nodes Visited 312
A*	Path Length 22 : 8	Nodes Visited 76
DIJKSTRA	Path Length 22 : 8	Nodes Visited 102

Performance overview
across algorithms

RESULT

Path finding behavior shows correct and efficient routes

Handles local optima without getting trapped

Execution observations confirm stable performance

Key metrics captured for each algorithm

Algorithm	Path Length	Time Complexity	Space Complexity	Stability
A*	Shortest	O(n log n)	O(n)	High
Dijkstra	Shortest	O(n^2)	O(n)	High
BFS	Shortest	O(n)	O(n)	High
DFS	Not Shortest	O(n)	O(n)	Low
Greedy	Not Shortest	O(n)	O(1)	Low